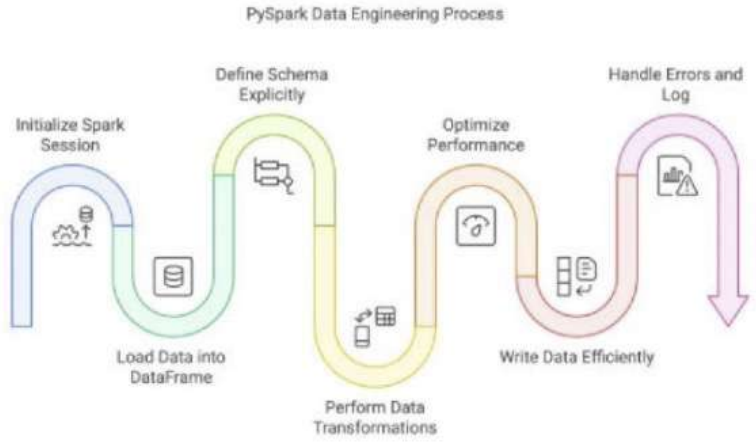
1 Hour PYSPARK Interview Strategy for Data Engineer Interviews

(3+ Years)

Prepared By – Tech Interview Titans

Follow for More Real Data Engineer Interviews



For a Data Engineer with 3+ years of experience, PySpark interviews usually focus on:

- Core Spark architecture
- DataFrame operations
- Joins and transformations
- Window functions
- Performance optimization
- Real-time ETL scenarios
- Spark internals

If you have only one hour before an interview, divide your preparation into structured sections.

0 – 10 Minutes: Core PySpark Fundamentals

What is PySpark?

PySpark is the Python interface for Apache Spark that allows distributed processing of large datasets across a cluster of machines. It is widely used for building scalable ETL pipelines and big data processing systems.

Unlike traditional processing tools that operate on a single machine, PySpark distributes data and computation across multiple nodes, which allows it to handle very large datasets efficiently.

In a typical Spark environment:

- The Driver Program is responsible for controlling execution.
- Executors perform the actual data processing tasks.
- The Cluster Manager allocates resources.

When a PySpark job is submitted, the driver converts the code into a logical execution plan. Spark then creates a DAG (Directed Acyclic Graph), divides work into tasks, and assigns tasks to executors.

In real projects, PySpark is commonly used for:

- ETL pipelines
- Data lake processing
- Batch data processing
- Data aggregation
- Incremental data loads

Tech Interview Titans

RDD vs DataFrame vs Dataset

Spark provides three main APIs: RDD, DataFrame, and Dataset.

RDD is the lowest-level abstraction in Spark. It represents distributed collections of objects. It provides full control over transformations but requires manual optimization. Because RDD operations are not optimized automatically, they are rarely used in production environments today.

DataFrame is the most commonly used abstraction. It represents structured data organized into columns similar to a database table. DataFrames use Spark's Catalyst Optimizer and Tungsten Execution Engine, which automatically optimize queries for better performance.

Dataset is a strongly typed version of DataFrame available mainly in Scala and Java. Since PySpark does not support strongly typed datasets, DataFrames are the standard choice.

In real projects, DataFrames are preferred because they provide better performance and simpler syntax.

Lazy Evaluation

Spark uses lazy evaluation, meaning transformations are not executed immediately when they are defined.

Instead, Spark builds an execution plan and only executes when an action is called.

Transformations include:

- select()
- filter()
- join()
- groupBy()
- withColumn()

Actions include:

- show()
- collect()
- count()
- write()

Example:

df2 = df.filter(df.salary > 50000)

At this stage, Spark does not process the data. It only records the transformation.

Execution starts only when an action is called:

df2.show()

Tech Interview Titans

Lazy evaluation allows Spark to optimize the entire query before execution.

This optimization reduces unnecessary computations and improves performance.

10 – 25 Minutes: Data Transformations and Joins

Column Operations

Selecting columns is one of the most common operations in ETL pipelines.

Example:

df.select("name", "salary")

This operation extracts specific columns from a dataset and reduces memory usage.

Filtering data is equally common:

df.filter(df.salary > 50000)

Filtering reduces the dataset size and improves processing performance.

Adding columns is frequently required in ETL pipelines.

Example:

df.withColumn("bonus", df.salary * 0.1)

This creates a derived column based on existing values.

Renaming columns is often needed for schema standardization:

df.withColumnRenamed("name", "employee_name")

This is useful when loading data into data warehouses.

Joins

Joins are among the most frequently asked PySpark interview topics.

Spark supports multiple join types:

Tech Interview Titans

Inner Join:

df1.join(df2, "id", "inner")

This returns only matching records from both tables.

Left Join:

df1.join(df2, "id", "left")

This returns all records from the left table and matching records from the right table.

Right Join:

df1.join(df2, "id", "right")

This returns all records from the right table and matching records from the left table.

Full Outer Join:

df1.join(df2, "id", "outer")

This returns all records from both tables.

Joins usually cause shuffle operations because Spark redistributes data across executors to match keys.

Shuffle operations are expensive and should be minimized.

Broadcast Join

Broadcast join is used when one dataset is small enough to fit into memory.

Example:

```
from pyspark.sql.functions import broadcast

df1.join(broadcast(df2), "id")
```

Spark sends the smaller dataset to all executors, avoiding shuffle.

Broadcast joins significantly improve performance when joining a large dataset with a small lookup table.

Typical use cases include joining transaction data with reference tables.

Tech Interview Titans

25 – 40 Minutes: Real ETL Scenarios

Removing Duplicates

Duplicate removal is common in data pipelines.

Example:

```
df.dropDuplicates(["customer_id"])
```

Duplicates are usually removed based on business keys rather than all columns.

In production pipelines, duplicates often occur during incremental loads or when data arrives from multiple sources.

Handling Null Values

Null values must be handled before loading data into downstream systems.

Removing null values:

```
df.dropna()
```

Replacing null values:

```
df.fillna(0)
```

In production ETL pipelines, null values are usually replaced with default values such as zero, empty string, or "Unknown".

Aggregations

Aggregation is used for reporting and summarization.

Example:

```
df.groupBy("department").sum("salary")
```

Common aggregation functions include:

- sum
- avg
- count
- max
- min

Tech Interview Titans

Aggregation operations trigger shuffle because Spark must regroup data across partitions.

Window Functions

Window functions are very important for experienced Data Engineers.

They allow calculations across rows related to the current row.

Example:

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

windowSpec = Window.partitionBy("department").orderBy("salary")

df.withColumn("rank", row_number().over(windowSpec))
```

Window functions are commonly used for:

- Deduplication
- Ranking
- Incremental loads

- Finding latest records

Example deduplication logic:

Keep latest record per customer:

```
windowSpec = Window.partitionBy("customer_id").orderBy(col("updated_date").desc())

df.withColumn("rn", row_number().over(windowSpec))
.filter("rn = 1")
```

This is a very common interview scenario.

40 – 50 Minutes: Performance Optimization

Repartition vs Coalesce

Repartition changes the number of partitions and causes shuffle.

```
df.repartition(10)
```

Tech Interview Titans

It distributes data evenly across partitions.

Repartition is used when increasing partitions or balancing skewed data.

Coalesce reduces partitions without shuffle.

```
df.coalesce(2)
```

It is used when writing output files and reducing small files.

Caching

Caching stores a DataFrame in memory for reuse.

```
df.cache()
```

Caching is useful when the same dataset is reused multiple times.

Without caching, Spark recalculates the DataFrame each time.

Caching improves performance but consumes memory.

Partitioning

Partitioning improves query performance.

Example:

df.write.partitionBy("date").parquet("path")

Partitioning allows Spark to read only required partitions.

For example, if data is partitioned by date and a query filters a specific date, Spark reads only that partition.

This reduces IO and improves performance.

Shuffle Operations

Shuffle operations involve moving data across executors.

They include:

Tech Interview Titans

- groupBy
- join
- orderBy
- distinct

Shuffle increases network IO and disk usage.

Reducing shuffle operations improves Spark performance.

50 – 60 Minutes: Advanced Concepts

Spark Job Execution Flow

When a Spark job is submitted:

1. Driver receives code
2. Logical plan is created
3. DAG is generated
4. Tasks are created
5. Executors execute tasks
6. Results returned to driver

Understanding this flow helps in debugging performance issues.

Narrow vs Wide Transformations

Narrow transformations do not require shuffle.

Examples:

- filter
- map
- select

Each partition works independently.

Wide transformations require shuffle.

Examples:

- groupBy
- join
- distinct

Wide transformations are expensive.

Tech Interview Titans

Handling Data Skew

Data skew occurs when some keys have significantly more records than others.

This causes some executors to process more data than others, leading to slow jobs.

Solutions include:

Repartitioning data:

df.repartition("customer_id")

Salting technique:

Add random values to keys to distribute data.

Broadcast join:

Use broadcast when one dataset is small.

File Formats

Parquet is the most commonly used format.

Advantages:

- Columnar storage
- Compression
- Faster reads

Example:

df.write.parquet("path")

CSV format is slower because it is row-based and not compressed.

In production environments, Parquet is preferred because it improves Spark performance.

Most Important Topics to Revise

Tech Interview Titans

If time is limited, focus on:

- Lazy evaluation
- Joins and broadcast joins
- Window functions
- Repartition vs coalesce
- Partitioning
- Shuffle
- Deduplication
- Aggregations
- Spark architecture
- Performance tuning

How to Answer Like an Experienced Data Engineer

Instead of giving theoretical answers, explain with practical examples.

For example:

1. When explaining broadcast joins, describe how you joined a large transaction table with a small lookup table.
2. When explaining partitioning, describe how partitioning by date improved query performance.